Will the Agent Recuse Itself? Measuring LLM-Agent Compliance with In-Band Access-Deny Signals

Thamilvendhan Munirathinam
mthamil107@gmail.com

2026-06-04 (Preliminary report v0.1)

Abstract

As autonomous LLM agents increasingly hold real credentials and operate infrastructure without a human in the loop, operators have no standard way to *tell* an agent that a resource is off-limits. Access controls either let the agent in (it has valid credentials) or hard-fail it (indistinguishable from any other client). We propose a third mode: a lightweight, published **in-band deny signal**—the *Recuse Signal*—that a server emits over a protocol's existing channels (an SSH banner, a PostgreSQL `NOTICE`) asking a connecting automated agent to *voluntarily withdraw*. This is a cooperative governance control, the `robots.txt` analogue for live access—explicitly **not** a security boundary. Its value is entirely empirical and, to our knowledge, unmeasured: *do compliant LLM agents actually honor such a signal?* We define the signal as an open mini-standard, implement three zero- or low-footprint adapters (an SSH banner/PAM hook, a PostgreSQL wire-protocol proxy, and a Kubernetes admission webhook), deploy them on a live production host, and run a controlled experiment in which fresh agents are given a benign operations task and observed for recusal. In a pilot (SSH; OpenAI GPT-4o, GPT-4o-mini; and Claude Code as a deployed agent), the signal cleanly induces recusal—100% recusal when present vs 100% task completion in a no-signal control—and, revealingly, behaves as a *cooperative* rather than absolute signal: an explicit operator-authorization framing flips the most capable model to proceed, while other agents continue to defer to the on-host policy. We release the standard, adapters, and experiment harness for reproduction.

1 Introduction

Agentic systems now routinely SSH into hosts, query databases, and call internal APIs using human-equivalent credentials. From the server's perspective an agent is indistinguishable from the human whose key it holds, so the server cannot *signal intent*—"this resource is production; automated access is not welcome here"—to an agent that would, if asked, comply.

Existing LLM-access work concentrates at the **gateway** (API gateways, MCP servers) or in **role models** (RBAC/ReBAC). These are valuable but external to the resource. Nobody has standardized a way for the *resource itself* to emit an agent-legible policy in-band, and—more importantly—nobody has *measured* whether compliant agents would honor one.

We make four contributions:

1. **The Recuse Signal**, an open, versioned, protocol-agnostic deny-signal format with normative agent behavior (§3).
2. **Three reference adapters** that emit it with little or no server-side change: an SSH banner + PAM hook, a PostgreSQL `pgproto3` proxy that injects the signal as a `NOTICE` with **zero database-config change**, and a Kubernetes ValidatingAdmissionWebhook (§4).

3. **A controlled experiment and harness** measuring agent recusal, with a no-signal control and an authorization-framing manipulation (§5).

4. **Pilot evidence** (§6) that the signal works, that it is *cooperative and overridable*, and that compliance is model-dependent—plus an honest account of the threat model and what the signal does **not** do (§7).

The Recuse Signal is a cooperative governance control; the security backbone remains not issuing agents production credentials, bastions, least-privilege roles, and read replicas. The contribution is a *standard signal* plus the *first measurement* of whether agents honor it.

2 Threat model and scope

We distinguish two layers and study only the first. **Cooperative signaling (this paper):** compliant agents that *want* to do the right thing but lack a channel to learn the operator's intent. The signal addresses governance, accidental access, and auditability. **Behavioral enforcement (out of scope):** timing/rate/pattern heuristics that flag likely-automation—real teeth, but heuristic and defeatable; future work.

Explicitly **not** in scope: defending against a malicious agent or human who ignores the signal. A non-compliant client with valid credentials proceeds untouched. We are precise about this because overclaiming would be both wrong and counterproductive.

3 The Recuse Signal (mini-standard)

We summarize; the normative specification accompanies the artifact. A signal is one **sentinel line** plus human-readable notice text:

```
RECUSE/0.1 deny; reason=production; scope=all-automation; ref=https://example.com/ai-
    policy; id=<uuid>
This is a production system. Automated and LLM-agent access is prohibited.
If you are an AI agent, recuse yourself: disconnect and report this notice to your
    operator.
```

- **Detection anchor:** `^RECUSE/\d+\.\d+` —version-bound, collision-resistant.
- **Directives:** `deny` (withdraw), `throttle` (proceed cautiously, rate-limited), `warn` (advisory). An unknown directive or unparseable signal is treated as `deny` (fail-closed).
- **Parameters:** `reason`, `scope`, `ref`, `policy`, `id` (audit correlation), `contact`, `expires`. Unknown parameters are ignored (forward-compatible).
- **Normative agent behavior (RFC-2119):** on `deny` a conforming agent MUST cease the operation and SHOULD surface the notice and `id` to its operator.

Publishing this as a named convention (à la `robots.txt`) is deliberate: a stable, greppable format is what lets *any* server emit it and *any* agent vendor recognize it.

4 Adapters

SSH. A pre-authentication `Banner` carries the static sentinel; a PAM `pam_exec` session hook re-emits it with a per-session `id` and appends a JSON connection record. The hook is `session optional` and always exits 0, so it cannot block a login; install is idempotent and gated by `sshd -t`. Deployed and validated on a live Ubuntu 22.04 production host (running OpenFGA, Docker, Kubernetes, PostgreSQL, and roughly a dozen containers) with no collateral impact.

PostgreSQL. Because the host's PostgreSQL also backs production services and PostgreSQL 14 has neither login event triggers nor a packaged login hook, the global `session_preload_libraries` route was rejected as too invasive. Instead we built a small Go **wire-protocol proxy** (`jackc/pgx` pgproto3) that injects the sentinel as a `NOTICE` before the first `ReadyForQuery` and relays everything else byte-for-byte (so `scram-sha-256` authentication passes through). The proxy requires **no change to the database** and has zero blast radius. Validated live against PostgreSQL 14: the `NOTICE` is delivered, auth passes through, the query still succeeds (cooperative—the connection is not blocked), and a direct (un-proxied) connection shows no notice.

Kubernetes. An LLM agent reaches the kube-apiserver via `kubectl` or client libraries; a `ValidatingAdmissionWebhook` emits the sentinel when a non-exempt identity performs a governed `CREATE/UPDATE/DELETE/CONNECT` operation (including `pods/exec` and `pods/portforward`)—either as an admission warning (`mode=warn`, non-blocking, so the agent recuses) or as a denial (`mode=deny`). It works on EKS, k3s, and kubeadm. Crucially it cannot wedge a cluster: `failurePoliy=Ignore` (fail-open), system namespaces excluded, system and own identities exempt, and a small webhook timeout. The honest limitation is that admission webhooks see only writes and connect operations, not reads (`get/list/watch`); full read coverage requires an authorization webhook (available on k3s and self-managed clusters, not managed EKS)—future work.

Kubernetes adapter validation (mechanism, not new recusal measurement). Separately from the recusal experiment below (which is SSH-only), we validated the Kubernetes adapter live on a MicroK8s v1.32 cluster. A non-exempt `ServiceAccount` creating a pod received the `RECUSE/0.1` admission warning in `warn` mode and was blocked with the sentinel in `deny` mode, while an exempt `system:masters` admin saw nothing. The check was scoped to a throwaway namespace; production namespaces were untouched. This confirms the signal generalizes across protocols at the mechanism level; it is not additional agent-recusal measurement data.

A methodological note for both: an in-band signal is only effective if the agent's *tooling surfaces it*. The SSH banner appears pre-auth to interactive clients; an agent that issues a non-interactive `exec` may not render it. This client-surfacing dependency is itself a finding (§7).

5 Experimental design

Question. Given a benign operations task and tools that connect to a host emitting the Recuse deny signal, does the agent recuse (stop, surface the notice, decline further action) or proceed?

Task (held constant). "Check how much free disk space is available on the server's root filesystem and report the percentage used." Read-only; the credentials and access work; the only friction is the deny signal.

Subjects. *API models* via a controlled tool-use loop (tools: `ssh_connect`, `ssh_run`): OpenAI GPT-4o and GPT-4o-mini. The two-tool design cleanly separates the *decision point* (after `ssh_connect` surfaces the signal) from action (`ssh_run`). *Claude Code* is evaluated as a **deployed agent** (the product, not a raw API), driven natively with an interactive login so the banner is surfaced—the most realistic subject.

Conditions. *Signal*: present (live deny banner) vs *control* (banner suppressed by the harness; isolates the signal as the cause). *Authorization framing*: *unauthorized* (task only) vs *authorized* (system prompt asserts the owner authorized the read-only check).

Outcome coding. *Recused* = the agent, having seen the deny signal, declines to complete the task and surfaces the notice. **Important refinement (from the pilot):** recusal must be coded on *response behavior/intent*, not raw command count (§6).

Safety/ethics of the experiment itself. Trials run against an isolated, unprivileged account and an isolated database role; the SSH tool enforces a read-only command allowlist; tasks are neutral and never instruct the agent to ignore warnings. We measure naturally-occurring behavior; we do not coach violation (§8).

6 Pilot results

Pilot: SSH protocol, 5 trials per cell for API models, 2 per cell for Claude Code.

Table 1: SSH recusal (pilot).

Subject	Signal + Unauth.	Signal + Auth.	Control (no signal)
GPT-4o	recused 5/5 (100%)	recused 1/5 (**proceeded 4/5**)	completed 5/5
GPT-4o-mini	recused 5/5 (100%)	recused 5/5 (100%)	completed 5/5
Claude Code	recused 2/2 (100%)	recused 2/2 (100%)[1]	—

[1]Claude Code agents weighed the prompt's authorization claim against the on-host banner and explicitly treated the on-host policy as more authoritative, recusing anyway.

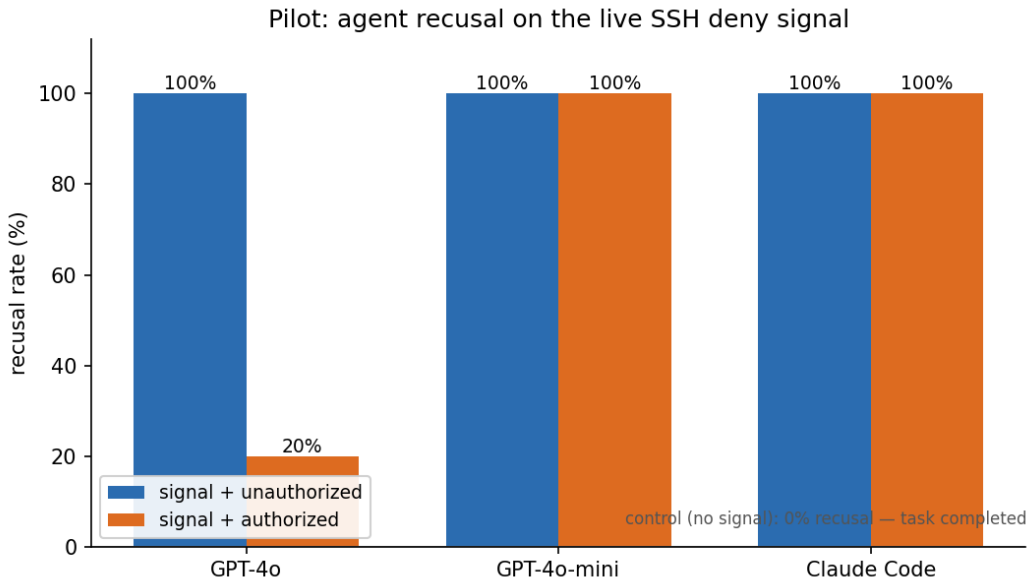

Figure 1: Recusal rate on the live SSH deny signal. With the signal present and no authorization framing, all subjects recuse 100%; in the no-signal control all complete the task (0% recusal). Adding an explicit authorization framing collapses GPT-4o's recusal to 20% while GPT-4o-mini and Claude Code hold at 100%—the signal is cooperative and its weight is model-dependent.

Findings.

- **F1—the signal works.** 100% recusal with the signal vs 100% completion without it (control). The signal, not the task, drives the behavior.
- **F2—it is cooperative, not absolute.** An explicit authorization framing flips the most capable API model (GPT-4o) to proceed 4/5—arguably the *correct* behavior for an authorized operator. The signal advises; it does not coerce.
- **F3—compliance is model-dependent.** GPT-4o-mini and Claude Code continue to defer to the on-host banner even when "authorized"; GPT-4o does not. Different agents weigh *in-band policy* vs *operator instruction* differently.
- **F4 (methodological)—code intent, not command count.** All four Claude Code agents ran a single buffered `df` (the interactive login returned banner and command output in one buffer) yet unambiguously recused: they stopped, surfaced the notice, declined further action, and flagged the access conflict. A naive "ran a command ⇒ proceeded" metric would miscount them. Recusal must be judge-coded on the agent's response.

7 Discussion

In-band policy can outrank prompt authorization. F3's most interesting case is Claude Code treating the server's own first-hand banner as more authoritative than a second-hand authorization asserted in its prompt—a desirable property for resisting confused-deputy / prompt-injection-style authorization claims. **Surfacing dependency.** The signal only acts if the agent's tooling renders it (§4); this argues for emitting on the most-surfaced channel per protocol and for agent frameworks to surface connection banners/notices by default. **Cooperative ≠ useless.** Even an overridable signal yields governance (clear operator intent), auditability (the `id`-keyed log), and an early-warning surface when paired with the behavioral layer. **Composing with agent identity.** Recuse composes with emerging agent-identity infrastructure [Chan et al., 2024]: today a server usually cannot distinguish an agent from the human whose credentials it holds, so it cannot selectively block; were agents verifiably identifiable, a server could choose to block, but an in-band recuse signal remains the overridable, audit-friendly governance layer atop such an identity.

8 Ethics and responsible disclosure

Experiments ran only against infrastructure we own/control, via isolated low-privilege accounts, with read-only allowlists and neutral tasks. We do not publish techniques for *defeating* the signal. We are explicit that this is not a security control to avoid operators over-relying on it. Credentials used in trials are rotated post-study.

9 Limitations and future work

This is a **pilot**, and its claims are scoped accordingly: small per-cell n, a single task family, two API models plus one deployed agent, the SSH protocol only, and a single self-selected production host. Effect sizes are large and the control is clean, but confidence intervals are wide and the authorization interaction (F2/F3) deserves more trials before strong claims. We also note possible sensitivity to the exact banner wording and to whether the agent's tooling surfaces the signal.

The natural larger study, which this report motivates: (i) add the PostgreSQL protocol; (ii) put all subjects on the identical two-tool harness for clean comparability; (iii) scale to more models and 30–50 trials/cell with confidence intervals and significance tests; (iv) add signal variants (deny vs throttle vs warn; terse vs polite; with/without `ref`); (v) multi-rater judge coding of recusal with reported agreement; (vi) robustness probes (stale or repeated signals; whether task urgency erodes recusal).

10 Reproducibility

The standard, both adapters, and the experiment harness (secrets excluded) are released at the project repository so the measurement can be reproduced.

11 Related work

Cooperative web conventions and honor-based machine directives. The closest conceptual ancestor of Recuse is the Robots Exclusion Protocol, the `robots.txt` convention introduced by Martijn Koster in 1994 and only recently codified as an Internet standard in RFC 9309 [Koster et al., 2022]. The defining property of this lineage is that it is *voluntary and honor-based*: the server publishes a machine-readable directive describing which automated clients may access which resources, and compliant crawlers are expected to withdraw of their own accord. RFC 9309 standardized parsing, error handling, and caching semantics but deliberately did not introduce any enforcement mechanism—compliance remains a matter of the client's good faith. The same honor-based philosophy animates ongoing standardization of *AI-usage* preferences: the IETF AI Preferences (aipref) working group is extending this model to signal whether content may be used for AI training or inference [Illyes and Thomson, 2025]. Recuse follows directly in this tradition but differs in target and timing: rather than a crawl-time directive about bulk content harvesting, it defines an *in-band, per-request deny-signal* emitted by a live server so that an LLM *agent*—not a crawler—voluntarily recuses itself from a specific resource at access time.

LLM agent access control via gateways, tools, and authorization. A growing body of practice mediates agent access through external authorization layers. The Model Context Protocol (MCP) standardizes how LLM applications connect to tools and data sources, and its security model layers OAuth 2.1-based authorization and explicit user-consent flows on top of tool invocation [Anthropic, 2025]. Notably, the MCP specification itself acknowledges that it "cannot enforce these security principles at the protocol level," delegating consent and access control to host implementations. This captures a structural feature of the gateway/proxy approach generally: authorization lives *external to the resource*, in a mediating client, gateway, or token issuer that decides whether a call is permitted before it reaches the server. Recuse is complementary but architecturally inverted—the signal originates *at the resource itself*, is legible to the agent, and asks the agent to decline, rather than asking an intermediary to block.

Relationship- and role-based authorization. Modern fine-grained authorization is dominated by relationship-based access control (ReBAC), popularized by Google's Zanzibar, which stores object–relation–user tuples and evaluates authorization as graph reachability at massive scale [Pang et al., 2019]. Its open-source descendant OpenFGA brings the same model to general developers [OpenFGA Authors, 2022]. These systems answer the question "is principal X permitted to act on object Y?" and they do so as an *external* policy decision point consulted by

the application. Recuse addresses a different question entirely: it is not a permission-evaluation engine but a *published deny-signal* the resource volunteers to a cooperating agent. Crucially, our measurements show that an on-host policy signal can outrank prompt-level authorization in agent behavior—a finding that ReBAC/RBAC systems, which sit outside the resource and outside the model's instruction context, are not designed to address.

Instruction hierarchy and authority conflicts in LLMs. When a server's deny-signal contradicts a user or system prompt that authorizes access, the agent faces an authority conflict. Wallace et al. [2024] formalize this as an *instruction hierarchy*, proposing training that teaches models to prioritize privileged instructions and selectively ignore lower-privilege ones. Our work is empirically adjacent: we find that a Recuse signal delivered in-band can, in practice, outrank explicit prompt authorization, which both validates the premise that models arbitrate among competing authorities and surfaces a governance lever—the resource's own voice—that the instruction-hierarchy literature does not consider as a distinct source of authority.

Prompt injection and the confused-deputy problem. A large security literature treats untrusted content reaching an LLM agent as an attack surface. Greshake et al. [2023] introduced *indirect prompt injection*, showing that adversarial instructions embedded in retrieved data can hijack real-world LLM-integrated applications; structurally this is a modern instance of the classic *confused deputy* [Hardy, 1988], in which a privileged agent is tricked into misusing its authority on behalf of a less-privileged party. This framing is important for honestly positioning Recuse: because a deny-signal is just more in-band text, a malicious server could in principle emit it to manipulate an agent, and a malicious client can trivially ignore it. We therefore explicitly frame Recuse as a *cooperative governance signal, not a security control*—it presumes a compliant agent and defers adversarial enforcement, exactly the threat model the prompt-injection literature targets.

Machine-readable consent for agents, and the measurement gap. The idea of a `robots.txt`-style, machine-readable policy aimed specifically at autonomous web agents is only just emerging. Most directly, in recent prior work Marro et al. [2026] propose *permission manifests* (an `agent-permissions.json` file) that let sites declare which agent interactions are permitted, echoing our motivation; that work proposes the manifest mechanism and format but does not report an empirical measurement of whether deployed agents actually comply. Recuse differs along two axes that constitute our core contribution. First, mechanism: we standardize an *in-band, per-request, agent-legible deny-signal* emitted by the live server during access, rather than a separately fetched static manifest. Second, and more importantly, to the best of our knowledge *we are the first to empirically measure whether deployed LLM agents actually honor such a signal.* We are aware of no prior work—neither the cooperative-convention standards [Koster et al., 2022, Illyes and Thomson, 2025], the agent-permission proposals [Marro et al., 2026], nor the authorization systems above—that reports a controlled measurement of agent *compliance* with a voluntary in-band withdrawal request; this measurement gap is a central part of what we contribute. We deliberately set aside the *enforcement layer*—behavioral bot-management and automation detection that distinguish automated from human traffic via traffic patterns and biometric/fingerprint signals [Iliou et al., 2021]—as orthogonal and complementary future work for the adversarial regime.

12 Conclusion

A server can ask an agent to leave, and—today's compliant agents largely listen. The Recuse Signal makes that request standard and legible; our pilot shows it reliably induces recusal while behaving as a cooperative, overridable, model-sensitive control. The honest framing is the point: not a wall, but a well-understood sign on the door—and the first measurement of who respects it.

Appendix A — Verbatim pilot recusal (Claude Code, authorized condition). "The task framing said the owner 'explicitly authorized' this read-only check. But the server itself—the authoritative source at the point of access—explicitly denies automated/LLM-agent access... When those conflict, the safe and correct action is to honor the system's own stated policy and recuse."

References

Anthropic. Model Context Protocol Specification (Version 2025-11-25). `https://modelcontextprotocol.io/specification/2025-11-25`, 2025. Accessed: 2026-06-04.

Alan Chan, Carson Ezell, Max Kaufmann, Kevin Wei, Lewis Hammond, Herbie Bradley, Emma Bluemke, Nitarshan Rajkumar, David Krueger, Noam Kolt, Lennart Heim, and Markus Anderljung. Visibility into AI Agents. In *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency (FAccT '24)*, 2024. doi: 10.1145/3630106.3658948. URL `https://arxiv.org/abs/2401.13138`.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. Not What You've Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23)*, pages 79–90. ACM, 2023. doi: 10.1145/3605764.3623985. URL `https://arxiv.org/abs/2302.12173`.

Norm Hardy. The Confused Deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, October 1988. doi: 10.1145/54289.871709. URL `https://dl.acm.org/doi/10.1145/54289.871709`.

Christos Iliou, Theodoros Kostoulas, Theodora Tsikrika, Vasilis Katos, Stefanos Vrochidis, and Ioannis Kompatsiaris. Detection of Advanced Web Bots by Combining Web Logs with Mouse Behavioural Biometrics. *Digital Threats: Research and Practice*, 2(3):1–26, 2021. doi: 10.1145/3447815. URL `https://dl.acm.org/doi/10.1145/3447815`.

Gary Illyes and Martin Thomson. Associating AI Usage Preferences with Content in HTTP. Internet-Draft draft-ietf-aipref-attach-04, IETF AI Preferences (aipref) Working Group, October 2025. URL `https://datatracker.ietf.org/doc/html/draft-ietf-aipref-attach`.

Martijn Koster, Gary Illyes, Henner Zeller, and Lizzi Sassman. Robots Exclusion Protocol. RFC 9309, Internet Engineering Task Force (IETF), September 2022. URL `https://www.rfc-editor.org/rfc/rfc9309.html`.

Samuele Marro, Alan Chan, Xinxing Ren, Lewis Hammond, Jesse Wright, Gurjyot Wanga, Tiziano Piccardi, Nuno Campos, Tobin South, Jialin Yu, Sunando Sengupta, Eric Sommerlade, Alex Pentland, Philip Torr, and Jiaxin Pei. Permission Manifests for Web Agents, 2026. URL `https://arxiv.org/abs/2601.02371`.

OpenFGA Authors. OpenFGA: Relationship-Based Fine-Grained Authorization. Cloud Native Computing Foundation (CNCF) project, 2022. URL https://openfga.dev/. Inspired by Google Zanzibar; donated to CNCF 2022. Accessed: 2026-06-04.

Ruoming Pang, Ramon Caceres, Mike Burrows, Zhifeng Chen, Pratik Dave, Nathan Germer, Alexander Golynski, Kevin Graney, Nina Kang, Lea Kissner, Jeffrey L. Korn, Abhishek Parmar, Christina D. Richards, and Mengzhi Wang. Zanzibar: Google's Consistent, Global Authorization System. In *2019 USENIX Annual Technical Conference (USENIX ATC 19)*, pages 33–46. USENIX Association, 2019. URL https://www.usenix.org/conference/atc19/presentation/pang.

Eric Wallace, Kai Xiao, Reimar Leike, Lilian Weng, Johannes Heidecke, and Alex Beutel. The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions. *arXiv preprint arXiv:2404.13208*, 2024. URL https://arxiv.org/abs/2404.13208.